Vittam Security Review

Billing software backend, public pages and admin dashboard · 27 Oct 2026 · Findings only, nothing fixed

> **Scope:** Express/MSSQL backend (`backend/src`), public storefront and table-QR order pages, admin dashboard, CI workflows and repository contents.
> **Method:** Code review from an attacker's point of view. No live exploit requests were sent. `npm audit` was run, and the SQL driver's GUID validation was read in `node_modules` .

#	Finding	Severity
1	Owner accounts can be cracked by trying common PINs across many phones	**High**
2	Sign-up skips OTP and has no real limit, enabling WhatsApp spam/phishing	**High**
3	Expired/suspended businesses keep full API access	**High**
4	Staff bill routes accept negative/NaN/huge quantities	**High**
5	Removing access doesn't end sessions	**Medium**
6	Phone numbers only unique within a business	**Medium**
7	Receipt PDF endpoint can overload the server	**Medium**
8	Admin side weakly protected	**Medium**
9	OTPs guessable from many IPs; phones can be flooded	**Medium**
10	Cashier can void finalized bills and set ₹0 prices	**Medium**
11	No security headers	**Medium**
12	Vulnerable dependencies (1 critical, 8 high)	**Medium**
13–20	Hardening items	**Low**

High

1. **Owner accounts can be cracked by trying common PINs across many phone numbers**

`backend/src/routes/auth.js:369-460`

- Login is a phone number plus a 4-digit PIN.
- `/api/login` returns `404 phone_not_found` for unknown phones, so an attacker can tell which phones have accounts. `/send-otp` leaks the same with 409/404.
- The login rate limit is keyed on IP + phone, so every new phone gets a fresh budget, and each account allows 10 wrong PINs before it locks.
- **Attack:** list which phones exist, then try the 10 most common PINs (1234, 0000, 1111…) on each. That realistically opens a sizeable share of accounts, including owners, who see all business data.
- **Side effect:** anyone who knows a staff member's phone can keep that account locked. After the first lockout `failed_attempts` is never reset, so every 15 minutes one wrong attempt locks it again.

2. **Sign-up skips OTP and has no real limit, so anyone can send WhatsApp spam or phishing from your sender**

`auth.js:57 · middleware/rateLimiter.js:576 · routes/customers.js:311 · whatsapp.js:516`

- `/register` never asks for the `verified_token` from `/verify-otp`, so the OTP step at sign-up can be skipped by calling the API directly.
- `registerLimiter` has `skipSuccessfulRequests: true`, so successful sign-ups are never counted and one IP can create unlimited businesses.
- Every trial gets `allow_customers=1` and `whatsapp_mode='api'`.
- **Attack:** register a trial, ring ₹1 bills to target phones (this creates customer rows), then use `/api/customers/message`. That sends free text with an attacker-chosen shop name through your WhatsApp Business template, 500 a day per fake business.
- **Likely outcome:** Meta bans your sender number and you pay the per-message cost.

3. **Expired, suspended or device-blocked businesses keep full API access**

`backend/src/routes/license.js (only reader of subscription state)`

- Subscription `status`, `expires_at` and `allow_mobile` / `allow_desktop` are only read in `license.js`. The app enforces them; the server never does.
- Login only checks `is_verified`.
- An expired trial, or a business you suspended for abuse, keeps working against the API with its existing tokens, or with fresh ones from login. That includes the WhatsApp sends in #2.

4. **Staff bill routes accept negative, zero, NaN or huge quantities**

`backend/src/routes/bills.js:854 (compare menuPricing.js:37)`

- `qty = parseFloat(i.quantity)` is never checked in `POST /bills`, `add-items` or `update-items`. The public order paths do check it.
- A negative line lowers the bill total, which lets a cashier pocket the difference.
- It also adds stock instead of removing it: `stock - (-5)` passes the `>= @qty` guard.
- It corrupts GSTR-1/3B figures.

Medium

5. **Removing someone's access doesn't end their session**

`auth.js (requireAuth) · routes/auth.js:323, 563`

- `requireAuth` never checks `is_active` or the user's current role. Access tokens last 8 hours.
- `/refresh` signs a new token with the role from the old token, without checking that the user still exists or is active. Refresh tokens never rotate and last 30 days.
- `/reset-pin` doesn't revoke existing sessions. An owner who resets their PIN after a takeover doesn't kick the attacker out.

6. **Phone numbers are only unique within a business**

`schema.sql:83 (UQ_users_business_phone) · routes/auth.js:383 · routes/staff.js:51`

- Any owner can add a "staff" member with someone else's phone number.
- Login picks `recordset[0]` with no `ORDER BY`, so the real user may be unable to log in, or may land in the wrong business.
- `reset-pin` updates every row with that phone, and account deletion picks an arbitrary business.

7. **The receipt PDF endpoint can be used to overload the server**

`backend/src/routes/receipt.js:22-53`

- It is unauthenticated and launches a full Edge browser on every request, with a 30-second timeout. It does this even when the token is invalid, so no real receipt is needed.
- The limit is 30 requests per minute per IP, so a handful of IPs can pin the server's CPU and memory.

8. **The admin side is weakly protected**

`routes/admin.js:113 · routes/license.js:152-224 · static/admin/index.html:313`

- `/admin/api/login` has only the global limit (1000 requests per 15 minutes per IP), no lockout and no MFA.
- The admin token is stored in `localStorage` on the same origin as the public store, order and receipt pages. A single XSS on any of those pages would hand over the platform admin token.
- The admin page loads `cdn.sheetjs.com` with no SRI hash, so a compromise of that CDN gives full admin.
- `/api/license/admin/activate` is protected by a static header secret compared with plain `!==` (timing leak), has no limiter, and returns `err.message` to the caller.

9. **OTPs can be guessed from many IPs, and the endpoints can be used to flood a phone**

`whatsapp.js:224 · middleware/rateLimiter.js:632`

- `otpVerifyLimiter` is keyed on phone + IP, so guesses spread across many IPs add up. There is no attempt counter per OTP.
- Forgot-PIN leads to owner takeover, so guessing an OTP is worth real money to an attacker.
- Sending is capped at 5 per IP per 15 minutes. `/send-otp` with purpose register, and every shop's store and order pages, can each be used to flood any phone with WhatsApp OTPs.

10. **A cashier can void finalized bills and set any price, including ₹0**

`routes/bills.js:2283 (DELETE /bills/:id) · price_override`

- This is classic till fraud. Check whether the owner actually sees these actions, rather than them just sitting in the audit table.

11. **No security headers**

`backend/src/server.js`

- There's no helmet: no CSP, HSTS, `X-Content-Type-Options` or `X-Frame-Options`, so the store and order pages can be framed (clickjacking). `X-Powered-By` is exposed.

12. **Vulnerable dependencies**

`backend/package.json · npm audit --omit=dev: 19 issues (1 critical, 8 high)`

- The critical one, `proxy-addr` (IP spoofing), matters here because every rate limit depends on `req.ip`.
- `body-parser` (direct dependency) has a high-severity denial-of-service issue.

Low / hardening

13. **OTP verification token accepted as a staff token**

`routes/auth.js:313`

The `verified_token` from `/verify-otp` is signed with `JWT_ACCESS_SECRET` and has no `typ`, so `requireAuth` and the WebSocket accept it as a staff token. It has no `business_id`, so the damage is limited today. It's the same mistake the code already fixed for store tokens.

14. **Startup checks only warn**

The startup checks for environment variables and weak secrets only log a warning; the server still boots. If production ever runs without `NODE_ENV=production` and with WhatsApp turned off, the API returns `dev_otp` in responses. That would let anyone take over any account through forgot-PIN.

15. **iisnode developer errors enabled**

`web.config` has `devErrorsEnabled="true"`, so iisnode shows stack traces and file paths to clients when the app crashes.

16. **Client-controlled log IDs**

The client-supplied `X-Request-Id` header is trusted as the log ID, so log entries can be forged.

17. **Low-privilege roles see customer data**

Kitchen and server roles can read the credit ledger, all bills and the customer search, which exposes customer PII to staff who don't need it.

18. **Repository hygiene**

`node_modules`, `backend/logs/app.1.log`, `bash.exe.stackdump` and `run-query.js` (an ad-hoc SQL runner) are committed. Also confirm that the Firebase API keys in `firebase_options.dart` are restricted in the Google Cloud console.

19. **Smaller gaps**

`/deletion/cancel` has no `otpVerifyLimiter`. The WebSocket token travels in the query string, so it ends up in IIS logs. The `order` OTP is not tied to a specific shop or table.

20. **Login responses reveal account state**

`attempts_remaining`, the "pending verification" message shown before the PIN is checked, and a 423 status for locked accounts.

Held up under attack

- All SQL is parameterized, and dynamic column lists are built from whitelists.
- The driver strictly validates GUIDs, so upload filenames can't be used for path traversal.
- Business-ID scoping is consistent across the routes traced.
- HTML output is escaped in the receipt, store and admin pages.
- OTPs use crypto-random values, are stored hashed and work only once.
- Receipt and QR tokens are long and random enough not to be guessed.
- App tokens are kept in `flutter_secure_storage`.

Quickest wins: #1, #2 and #4. A few lines each would close most of the real-world risk.